



Start here x Sllo.c x

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *next;
7  };
8
9  struct Node* createNode(int data) {
10     struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
11     newNode->data = data;
12     newNode->next = NULL;
13     return newNode;
14 }
15
16 void insertEnd(struct Node **head, int data) {
17     struct Node *newNode = createNode(data);
18     if (*head == NULL) {
19         *head = newNode;
20     } else {
21         struct Node *temp = *head;
22         while (temp->next != NULL)
23             temp = temp->next;
24         temp->next = newNode;
25     }
26 }
27
28 void display(struct Node *head) {
29     if (head == NULL) {
30         printf("List is empty\n");
31     } else {
32         struct Node *temp = head;
33         while (temp != NULL) {
34             printf("%d ", temp->data);
35             temp = temp->next;
36         }
37         printf("\n");
38     }
39 }
```



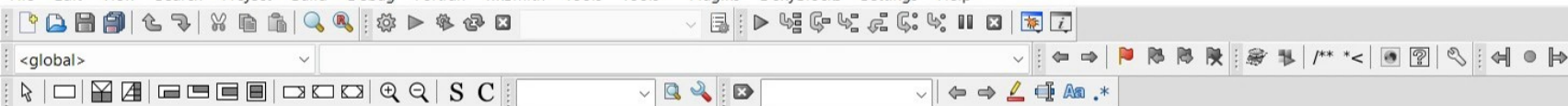
Start here x Silo.c x

```
40
41 void sortList(struct Node *head) {
42     if (head == NULL) return;
43     struct Node *current = head, *index = NULL;
44     int temp;
45
46     while (current != NULL) {
47         index = current->next;
48         while (index != NULL) {
49             if (current->data > index->data) {
50                 temp = current->data;
51                 current->data = index->data;
52                 index->data = temp;
53             }
54             index = index->next;
55         }
56         current = current->next;
57     }
58 }
59
60 struct Node* reverseList(struct Node *head) {
61     struct Node *prev = NULL, *current = head, *next = NULL;
62
63     while (current != NULL) {
64         next = current->next;
65         current->next = prev;
66         prev = current;
67         current = next;
68     }
69     return prev;
70 }
71
72 struct Node* concatenate(struct Node *head1, struct Node *head2) {
73     if (head1 == NULL) return head2;
74     struct Node *temp = head1;
75     while (temp->next != NULL)
76         temp = temp->next;
77     temp->next = head2;
78     return head1;
79 }
```



Start here x Silo.c x

```
79 }
80
81 int main() {
82     struct Node *list1 = NULL, *list2 = NULL;
83     int choice, value, list_choice;
84
85     while (1) {
86         printf("\nMenu:\n");
87         printf("1. Insert into List 1\n");
88         printf("2. Insert into List 2\n");
89         printf("3. Display Lists\n");
90         printf("4. Sort List\n");
91         printf("5. Reverse List\n");
92         printf("6. Concatenate Lists\n");
93         printf("7. Exit\n");
94         printf("Enter your choice: ");
95         scanf("%d", &choice);
96
97         switch (choice) {
98             case 1:
99                 printf("Enter value to insert in List 1: ");
100                 scanf("%d", &value);
101                 insertEnd(&list1, value);
102                 break;
103             case 2:
104                 printf("Enter value to insert in List 2: ");
105                 scanf("%d", &value);
106                 insertEnd(&list2, value);
107                 break;
108             case 3:
109                 printf("List 1: ");
110                 display(list1);
111                 printf("List 2: ");
112                 display(list2);
113                 break;
114             case 4:
115                 printf("Sort which list (1 or 2)? ");
116                 scanf("%d", &list_choice);
117                 if (list_choice == 1)
```



Start here x Silo.c x

```
105         scanf("%d", &value);
106         insertEnd(&list2, value);
107         break;
108     case 3:
109         printf("List 1: ");
110         display(list1);
111         printf("List 2: ");
112         display(list2);
113         break;
114     case 4:
115         printf("Sort which list (1 or 2)? ");
116         scanf("%d", &list_choice);
117         if (list_choice == 1)
118             sortList(list1);
119         else
120             sortList(list2);
121         break;
122     case 5:
123         printf("Reverse which list (1 or 2)? ");
124         scanf("%d", &list_choice);
125         if (list_choice == 1)
126             list1 = reverseList(list1);
127         else
128             list2 = reverseList(list2);
129         break;
130     case 6:
131         list1 = concatenate(list1, list2);
132         list2 = NULL;
133         printf("Lists concatenated. List 1 now contains both lists.\n");
134         break;
135     case 7:
136         exit(0);
137     default:
138         printf("Invalid choice. Try again.\n");
139     }
140 }
141 return 0;
142 }
143
```

Menu:

1. Insert into List 1
2. Insert into List 2
3. Display Lists
4. Sort List
5. Reverse List
6. Concatenate Lists
7. Exit

Enter your choice: 1

Enter value to insert in List 1: 45

Menu:

1. Insert into List 1
2. Insert into List 2
3. Display Lists
4. Sort List
5. Reverse List
6. Concatenate Lists
7. Exit

Enter your choice: 1

Enter value to insert in List 1: 34

Menu:

1. Insert into List 1
2. Insert into List 2
3. Display Lists
4. Sort List
5. Reverse List
6. Concatenate Lists
7. Exit

Enter your choice: 2

Enter value to insert in List 2: 345

Menu:

1. Insert into List 1
2. Insert into List 2
3. Display Lists
4. Sort List
5. Reverse List
6. Concatenate Lists

```
7. Exit
Enter your choice: 2
Enter value to insert in List 2: 36
```

```
Menu:
1. Insert into List 1
2. Insert into List 2
3. Display Lists
4. Sort List
5. Reverse List
6. Concatenate Lists
7. Exit
Enter your choice: 2
Enter value to insert in List 2: 156
```

```
Menu:
1. Insert into List 1
2. Insert into List 2
3. Display Lists
4. Sort List
5. Reverse List
6. Concatenate Lists
7. Exit
Enter your choice: 3
List 1: 45 34
List 2: 345 36 156
```

```
Menu:
1. Insert into List 1
2. Insert into List 2
3. Display Lists
4. Sort List
5. Reverse List
6. Concatenate Lists
7. Exit
Enter your choice: 5
Reverse which list (1 or 2)? 2
```

```
Menu:
1. Insert into List 1
2. Insert into List 2
```

```
E:\Monisha\BMSCE\SEM-III\D  ×  +  ▾  
5. Reverse List  
6. Concatenate Lists  
7. Exit  
Enter your choice: 6  
Lists concatenated. List 1 now contains both lists.  
  
Menu:  
1. Insert into List 1  
2. Insert into List 2  
3. Display Lists  
4. Sort List  
5. Reverse List  
6. Concatenate Lists  
7. Exit  
Enter your choice: 4  
Sort which list (1 or 2)? 1  
  
Menu:  
1. Insert into List 1  
2. Insert into List 2  
3. Display Lists  
4. Sort List  
5. Reverse List  
6. Concatenate Lists  
7. Exit  
Enter your choice: 3  
List 1: 34 36 45 156 345  
List 2: List is empty  
  
Menu:  
1. Insert into List 1  
2. Insert into List 2  
3. Display Lists  
4. Sort List  
5. Reverse List  
6. Concatenate Lists  
7. Exit  
Enter your choice: 7  
  
Process returned 0 (0x0)    execution time : 94.358 s  
Press any key to continue.
```